

Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Anonymous Authors

Abstract—Long-horizon robotic assembly requires a planning interface that preserves structural intent without overwhelming downstream geometric solvers. Purely semantic planners lack physical grounding, whereas directly applying native Logic-Geometric Programming (LGP) to the full assembly graph leads to excessive symbolic branching. We propose VLM-LGP, a graph-guided symbolic-geometric pipeline in which a Vision-Language Model first parses the target specification into an object-support graph and a decomposition module then converts this graph into solver-facing batches. Instead of relying on an ad hoc rule-based batching scheme, we formulate branch discovery as a multilevel graph partitioning problem and follow it with hierarchy-aware batch cutting to recover a dependency-safe execution order. The resulting decomposition is more interpretable than manual branch propagation because branch aggregation and execution cutting are handled explicitly as two different stages. Native LGP is instantiated only for the current batch, while waypoint-level screening and active collision constraints retain geometric feasibility. A representative assembly case shows how the support graph is first aggregated into the branch groups 134 | 256 | 789 and then cut into the execution sequence 1 | 34 | 2 | 56 | 7 | 8 | 9.

Index Terms—robotic assembly, logic-geometric programming, vision-language models, graph partitioning, task and motion planning

I. Introduction

LONG-horizon robotic assembly requires simultaneously deciding what structural relations should be realized and how they can be executed under geometric constraints. This coupling is difficult because assembly plans are inherently relational, while robot execution remains grounded in kinematics, contacts, and collisions. If the structural plan is underspecified, the robot receives commands that are logically plausible but physically infeasible. If the full geometric solver is exposed to the entire long-horizon task too early, the search space quickly becomes intractable.

Classical task and motion planning pipelines address part of this problem by separating symbolic planning from continuous motion generation [12]. This separation improves modularity, but it also creates a burden of manual specification: symbolic domains must be crafted carefully enough to encode the preconditions, effects, and object relations needed for the full assembly horizon. Recent language-conditioned planning systems reduce this manual effort by using large multimodal models as semantic front-ends. However, these systems still struggle to explain how a long structural goal should be decomposed into solver-consistent geometric subproblems.

Native Logic-Geometric Programming (LGP) provides a principled way to couple symbolic decisions with trajectory optimization [1]. Its strength is that symbolic backtracking is informed by geometric feasibility instead of being driven purely by abstract action templates. Nevertheless, directly applying native LGP to a full assembly graph exposes two practical bottlenecks. First, long-horizon tasks produce excessive symbolic branching, because many geometric dead ends are discovered only after expensive continuous reasoning. Second, assemblies with multi-support structures introduce topological patterns that are difficult to manage when the solver is forced to reason over the entire structure at once.

This paper revises the VLM-LGP story around a clearer structural interface. A Vision-Language Model is used to parse the target assembly into an object-support graph. The graph is then decomposed in two stages. We first apply multilevel graph partitioning to aggregate structurally coherent branches. We then apply hierarchy-aware batch cutting to turn each aggregated branch into dependency-safe execution batches that remain compatible with the current solver scale. The downstream LGP backend is instantiated selectively for one batch at a time, which yields a more interpretable and analytically justified decomposition than the previous rule-based batching logic.

The contributions of this draft are threefold. First, we formulate the structural interface between VLM parsing and LGP solving as an explicit object-support graph. Second, we replace rule-based branch propagation with a two-stage decomposition procedure based on multilevel graph partitioning and hierarchy-aware batch cutting. Third, we clarify how this decomposition feeds selective native LGP instantiation by exposing only the symbolic relations required by the current batch.

II. Related Work

Long-horizon robotic planning is traditionally studied through symbolic planning and task and motion planning abstractions. Languages such as PDDL [12] provide clean symbolic formalisms, but they depend on manually defined operators and do not by themselves resolve geometric feasibility. Logic-Geometric Programming closes this gap by coupling symbolic task structure and continuous optimization in a unified framework [1]. Follow-up efforts, including reactive or hierarchical variants, aim to improve robustness and receding-horizon behavior while preserving the symbolic-geometric coupling [4], [5].

Vision-language models have recently been explored as semantic interfaces for robot planning. Their appeal lies in reducing the burden of hand-specifying symbolic domains and allowing target structures or goals to be described more naturally. However, such models still require a disciplined interface to downstream geometric reasoning. Without an intermediate structural representation, the decomposition from a high-level target specification to a sequence of solver-consistent subproblems remains difficult to justify analytically [6].

Our work therefore draws from graph decomposition rather than from direct sequence-generation strategies. In particular, we use multilevel graph partitioning [3] as the algorithmic backbone for branch aggregation. This choice is motivated by its explicit coarsen-partition-refine procedure and by its ability to produce partitions that preserve strong internal connectivity while minimizing unnecessary cross-partition cuts. We adapt this idea to support graphs and combine it with hierarchy-aware batch cutting so that the final output is not just a structural partition, but an ordered sequence of batches that can be consumed by native LGP.

III. Preliminaries and Problem Statement

We represent a target assembly as an object-support graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $v \in \mathcal{V}$ denotes one target object instance and each directed edge $(u, v) \in \mathcal{E}$ means that object u must support object v in the final assembly. The graph is directed because support precedence is asymmetric: if $(u, v) \in \mathcal{E}$, then object u must be realized before object v can be stably placed. Optional edge attributes, such as left/right placement labels at the support interface, are kept as local geometric hints but do not by themselves define the batch sequence.

The planning goal is to convert $\mathcal{G}$ into an ordered sequence of solver-facing batches $\mathcal{B} = (B_1, \dots, B_T)$ such that every object appears exactly once, every batch remains within the current solver scale, and every support predecessor of a node in B_t has already appeared in an earlier batch. In other words, the system must produce a decomposition that is both structurally valid and suitable for downstream geometric solving.

Directly exposing native LGP to the full graph introduces two practical bottlenecks. First, long-horizon assemblies create excessive symbolic branching, because the solver must consider many candidate symbolic configurations before geometric infeasibility can prune them. Second, mixed support patterns such as bridge-like substructures complicate the symbolic-geometric search because local support changes can affect distant parts of the same full-horizon problem. These issues motivate a decomposition stage between graph extraction and native LGP instantiation.

IV. Method

Long-horizon robotic assembly requires both structural reasoning over target configurations and geometric reasoning over executable motions. The revised VLM-LGP

pipeline therefore uses the support graph as an intermediate structural interface between semantic parsing and solver instantiation. The pipeline proceeds in five steps: scene grounding, support-graph construction, multilevel graph partitioning for branch aggregation, hierarchy-aware batch cutting, and selective native LGP solving. The key change in this draft is that branch discovery and batch generation are no longer entangled in one hand-written rule set.

A. Scene Grounding and Feasibility Screening

Before structural planning, the system converts the initial scene into a grounded object inventory that records object category, geometric signature, and robot-centered identity. In the same grounding stage, it also constructs a deterministic scene-order dictionary by sorting objects of the same category according to their Euclidean distance to the robot base. This dictionary is computed once from the initial scene and later reused during solver-side instantiation to bind generic object requests to concrete scene instances; it is therefore independent of the target support graph and does not define the assembly strategy itself.

To remove infeasible candidates before long-horizon planning, the system queries the waypoint-level LGP subproblem [1] as a low-cost feasibility test. For each candidate object o_i , a sparse pick-waypoint problem is solved under collision constraints within a fixed time budget T_{max} :

$$\text{Reach}(o_i) = \mathbb{I}[\exists q^* \text{ s.t. } \text{KOM}_{\text{wp}}(o_i, q^*) \text{ is feasible}] \quad (1)$$

Objects that admit a feasible waypoint are retained in the grounded inventory, whereas failed or timed-out candidates are excluded from subsequent planning. In this way, the pipeline enters long-horizon planning with a solver-consistent feasibility prior obtained directly from the waypoint-level LGP subproblem, while avoiding the cost of full symbolic-geometric optimization at this stage.

B. Support Graph Construction

Instead of asking the VLM to directly output an executable manipulation sequence, we use it to parse the target specification into the object-support graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each node denotes one target object instance, and each directed edge denotes one direct support relation that must hold in the assembled structure. This graph captures what structural relations should be realized while deliberately postponing how they will be executed.

The support graph is the central interface of the pipeline. Upstream, it converts a semantic assembly description into an explicit structural representation. Downstream, it provides the input to the decomposition module that prepares solver-consistent subproblems for native LGP. Figure 2 shows representative support graphs parsed from target specifications.

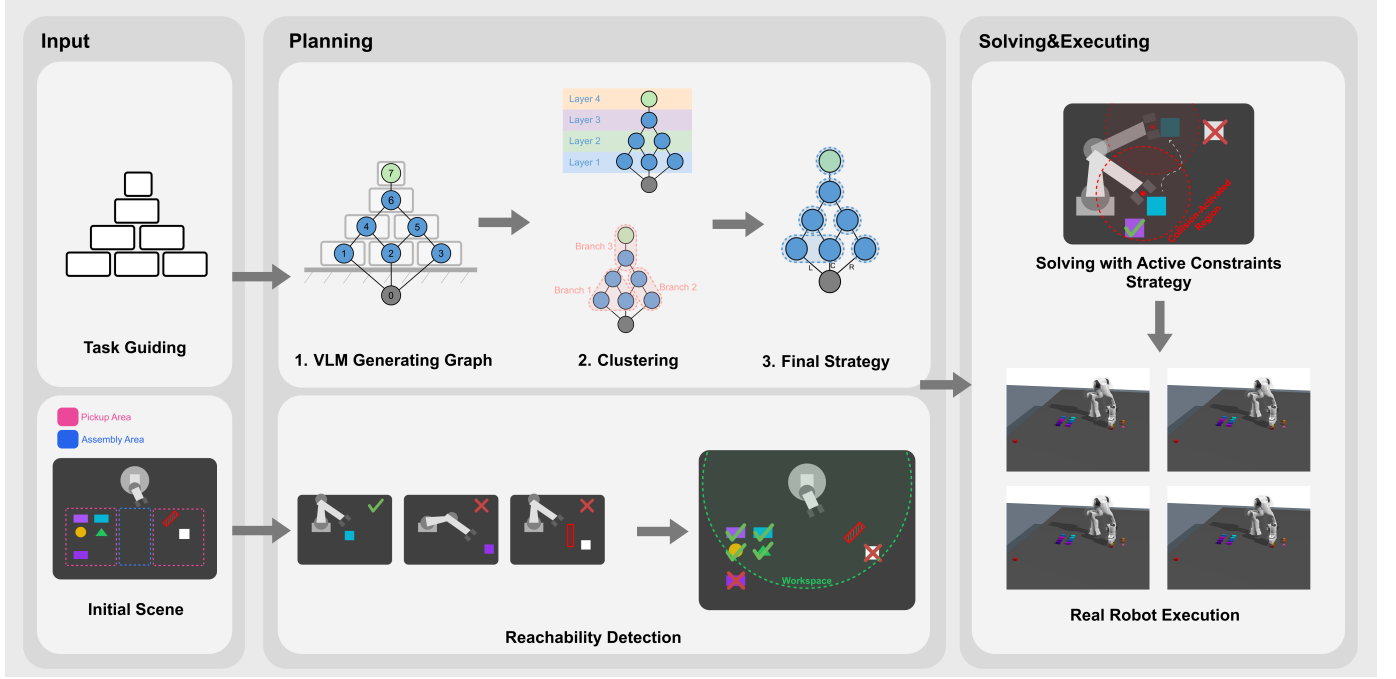


Fig. 1. The overall workflow of the proposed VLM-LGP framework.

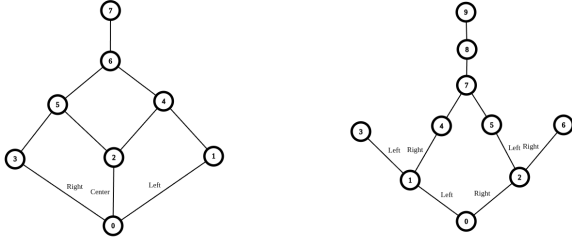


Fig. 2. Examples of object-support graphs parsed by the VLM from target assembly specifications. Directed edges encode direct support relations that are later used for branch aggregation and hierarchy-aware batch cutting.

C. Multilevel Graph Partitioning for Branch Aggregation

A direct topological batching of $\mathcal{G}$ tends to fragment the structure too early, because branch discovery and execution ordering become entangled in the same local rule set. We instead separate these two tasks. The first task is branch aggregation: identifying which nodes should be viewed as belonging to the same structural branch before any solver-facing batches are emitted.

To this end, we derive an auxiliary weighted graph $\bar{\mathcal{G}} = (\mathcal{V}, \bar{\mathcal{E}}, W)$ from the directed support graph. For branch aggregation, edge orientation is temporarily ignored, but the original directed support relation is preserved for the later precedence stage. The edge weight w_{uv} encodes structural affinity. Edges inside a single-support chain are assigned stronger affinity, whereas edges entering a multi-support transition are assigned lower affinity so that structurally merged regions can start a new branch. We

then solve a multilevel graph partitioning problem [3] to obtain a branch set $\mathcal{P} = \{P_1, \dots, P_m\}$:

$$\min_{\mathcal{P}} \sum_{(u,v) \in \bar{\mathcal{E}}} w_{uv} \mathbb{I}[\pi(u) \neq \pi(v)] + \lambda \sum_{j=1}^m ||P_j| - \bar{s}|, \quad (2)$$

where $\pi(v)$ denotes the partition index of node v , $\bar{s}$ is a target branch scale, and λ balances cut quality against partition size. The multilevel procedure follows the standard coarsen-partition-refine pattern: it first coarsens the graph, partitions the coarse graph, and then uncoarsens and refines the result on progressively finer graphs.

On the representative support graph used throughout this draft, the aggregation stage identifies three branch-level groups: the left support branch 134, the right support branch 256, and the upper bridge chain 789. This intermediate result is not yet an execution sequence. Its purpose is to recover coherent structural regions before we decide how to cut them into solver-facing batches.

D. Hierarchy-Aware Batch Cutting

After branch aggregation, each branch subgraph is converted into execution batches through hierarchy-aware cutting. This second stage is responsible for the actual solver-facing decomposition. It operates on the original directed support graph so that every emitted batch remains consistent with precedence.

For each branch subgraph P_j , we recover its induced directed support graph and compute a local depth for every node. The branch is then scanned from lower layers to higher layers, and nodes are grouped into local batches according to three rules: (i) every predecessor of a node in the current batch must already have been realized, (ii)

the current batch must remain within the solver scale, and (iii) nodes from the same local layer are grouped together only when they play the same immediate structural role inside the branch. This yields an execution sequence that is still grounded in the original hierarchy of the branch rather than in an arbitrary topological order.

In the running example, hierarchy-aware cutting transforms the aggregated branches into the local sequences $134 \mapsto 1 \mid 34$, $256 \mapsto 2 \mid 56$, and $789 \mapsto 7 \mid 8 \mid 9$. A precedence-consistent ordering over the aggregated branch graph then produces the final global execution sequence $1 \mid 34 \mid 2 \mid 56 \mid 7 \mid 8 \mid 9$. This two-stage decomposition is easier to explain than the previous rule-based batching scheme because branch discovery and execution cutting are handled explicitly as different optimization problems.

E. Selective Native LGP Instantiation and Solving

Native LGP couples symbolic task structure with continuous trajectory optimization through a shared representation of predicates, action semantics, and decision rules [1]. In principle, this abstraction can solve assembly tasks from symbolic goals alone. In long-horizon settings, however, directly exposing the full generic task domain still leads to excessive symbolic branching and unnecessary geometric backtracking.

Our solver therefore instantiates the native LGP abstraction selectively for each execution batch. Once a batch has been determined by the support graph, the system reads the scene-order dictionary generated during scene grounding, binds generic object categories to concrete scene instances, and compiles only the symbolic relations, object bindings, and decision rules required by the current support pattern. The solver thus receives a grounded symbolic-geometric subproblem tailored to the current batch rather than a monolithic domain covering all possible assembly alternatives.

The same waypoint-level solve also provides geometric cues for downstream motion optimization. Let x denote the full-motion trajectory optimized in the downstream stage, let $d_{ij}(x)$ denote the minimum pairwise distance between robot link l_i and object o_j along x , and let l_i^w denote the pose of link l_i at waypoint w . The downstream trajectory is obtained by solving

$$x^* = \arg \min_x J(x), \quad (3)$$

subject to collision constraints instantiated only for the active set

$$c_{ij}(x) = d_{\text{safe}} - d_{ij}(x) \leq 0, \quad \forall (l_i, o_j) \in \mathcal{C}_{\text{active}}(w), \quad (4)$$

where

$$\mathcal{C}_{\text{active}}(w) = \{(l_i, o_j) \mid d(l_i^w, o_j) < r_{\text{active}}\}. \quad (5)$$

Here, $J(x)$ denotes the downstream motion objective, d_{safe} is the prescribed safety margin, and r_{active} is the activation radius used to screen candidate collision pairs from waypoint geometry. Only pairs selected by $\mathcal{C}_{\text{active}}(w)$ are

TABLE I
Comparison of decomposition strategies on the representative support graph.

Strategy	Branch-level view	Final batch sequence
Previous rule-based batching	not explicit	1 2 34 56 7 8 9
Proposed two-stage decomposition	134 256 789	1 34 2 56 7 8 9

instantiated as explicit collision constraints in the downstream full-motion solve. This active-constraint strategy reduces unnecessary collision checks and shortens full-motion solve time without altering the structural plan itself.

This restricted instantiation preserves the expressiveness of native LGP while concentrating the search on the structurally relevant portion of the task. It also provides a clean interface between graph-level decomposition and geometric solving: the decomposition module decides which support relations should be realized next, and native LGP resolves whether and how those relations can be executed under the current scene geometry.

F. Robot Execution

After each subproblem is solved, the resulting trajectory is parsed and dispatched to the robot controller for execution. The execution layer remains intentionally thin: it reads the planned motion and gripper commands, executes them on the real platform, and updates the scene state for the next planning-solving cycle.

V. Experimental Results

A. Representative Decomposition Case Study

We use the representative support graph in Figure 2 as a running case to clarify the decomposition logic. The previous rule-based batching scheme emits the sequence $1 \mid 2 \mid 34 \mid 56 \mid 7 \mid 8 \mid 9$. In contrast, the revised method first aggregates the graph into the structural groups $134 \mid 256 \mid 789$ and only then cuts these groups according to local hierarchy, producing $1 \mid 34 \mid 2 \mid 56 \mid 7 \mid 8 \mid 9$. The difference is not merely a reordering. The revised method keeps structurally coherent branches intact during the aggregation stage and delays fine-grained cutting until the hierarchy of each branch has been made explicit.

B. Evaluation Focus of the Current Draft

The current draft focuses on explaining the decomposition logic rather than on claiming finalized quantitative gains. The intended evaluation protocol compares the previous rule-based batching strategy and the revised two-stage decomposition in terms of solver-facing fragmentation, the number of cross-batch support edges, and downstream LGP solve behavior. These quantitative results will be reported after the final execution pipeline and visual assets have been aligned with the revised method description.

VI. Conclusion

This draft reframes VLM-LGP around a clearer decomposition story for long-horizon robotic assembly. Instead of using a hand-written rule-based batching scheme, the method now separates structural decomposition into two explicit stages: multilevel graph partitioning for branch aggregation and hierarchy-aware batch cutting for execution ordering. This change makes the interface between support-graph parsing and native LGP instantiation easier to explain, easier to analyze, and better aligned with the current representative example. Future revisions will extend the experimental section with quantitative evidence that compares this revised decomposition against the previous rule-based pipeline.

References

- [1] M. Toussaint, “Logic-geometric programming for multi-step manipulation,” in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
- [2] M. Toussaint, “Newton methods for k-order Markov optimization,” *arXiv preprint arXiv:1407.0414*, 2014.
- [3] G. Karypis and V. Kumar, “Multilevel k-way partitioning scheme for irregular graphs,” *Journal of Parallel and Distributed Computing*, vol. 48, no. 1, pp. 96–129, 1998.
- [4] J. Schoeler, F. T. Rehder, and M. Toussaint, “R-LGP: Reactive Logic-Geometric Programming,” *arXiv preprint arXiv:2310.02791*, 2023.
- [5] J. Doe et al., “Receding Horizon Hierarchical Logic-Geometric Programming,” *arXiv preprint arXiv:2110.03420*, 2021.
- [6] F. Lin et al., “Text2Motion: From Natural Language to Logic-Geometric Programming,” *arXiv preprint arXiv:2303.12153*, 2023.
- [7] General VLA Reference, “Vision-Language-Action Models for Robotics,” Placeholder, 2024.
- [8] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed., 1999.
- [9] M. Toussaint et al., “General VLA Reference for Task and Motion Planning,” *arXiv preprint arXiv:2410.02193*, 2024.
- [10] J. Doe et al., “Vision-Language Models for Robotic Assembly,” *arXiv preprint arXiv:2407.09829*, 2024.
- [11] Y. Zhu et al., “Fabrica: Dual-Arm Assembly of General Multi-Part Objects via Integrated Planning and Learning,” *arXiv preprint arXiv:2506.05168*, 2024.
- [12] D. McDermott et al., “PDDL-the planning domain definition language,” *Technical Report*, Yale Center for Computational Vision and Control, 1998.